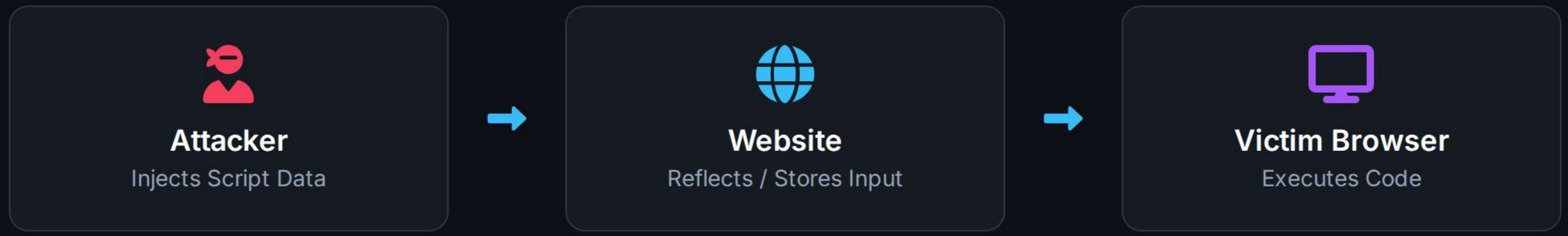
What If a Comment Could Run Code?

Cross-Site Scripting — When Data Becomes Code in the Browser



SPEAKER NOTES & INTERACTIVITY

- **Opening Hook Question:** "If I put JavaScript into a comment box, who do you think executes it?"
- **Core Concept:** Unlike SQLi (which targets the server), XSS targets the victim's web browser.

THE BASICS

What Does a Website Normally Do?

1. User
Submits text

2. Web App
Processes input

3. HTML
Generates page

4. Browser
Renders layout

COMMENT SUBMITTED:
`"Nice website!"`

BROWSER OUTPUT:
Nice website!

Takeaway: The browser receives content and decides how to interpret it.

SPEAKER NOTES & INTERACTIVITY

- **Explanation:** Browsers render text automatically. HTML structures text; JavaScript adds interactivity.
- **Key Insight:** Browsers trust whatever HTML content the website sends back to them.

THE VULNERABILITY

Where Does XSS Happen?



User Input

Untrusted string text

→

Boundary Collapse

Data treated as executable script

”XSS happens when attacker-controlled input is interpreted as executable content by a victim’s browser.”

SPEAKER NOTES & INTERACTIVITY

- **Simple Definition:** Explain XSS without jargon—web application fails to separate text data from web code.
- **Boundary Question:** Ask: "What happens if user input contains HTML tags like ?"

EXPLOITATION

The Injection Payload

ATTACKER ENTERS PROOF-OF-CONCEPT:

```
alert("XSS")
```

1. Submit
Attacker enters input

2. Store / Reflect
Website outputs input

3. Load
Victim opens page

4. Execute
JS runs in browser

Audience Question: Did the attacker directly hack the victim's computer? (No — the victim's browser executed content delivered through the website.)

SPEAKER NOTES & INTERACTIVITY

- **Audience Check:** Pause at the question. Highlight that the website acted as the unwitting carrier.
- **Safety Note:** Emphasize `alert()` is just a harmless test to prove JavaScript code execution.

CORE INSIGHT

Why Does This Work?

SAFE DATA

```
Hello everyone!
```

Browser displays plain text.

UNSAFE INPUT

```
alert("XSS")
```

Browser interprets executable JavaScript.

”The problem isn’t JavaScript itself.
The problem is untrusted input being treated as code.”

SPEAKER NOTES & INTERACTIVITY

• **Mental Model:** Browsers cannot guess programmer intentions—they only parse HTML syntax tags.
• **Key Point:** ` 4. Observe Behavior Popup executes on page load LIVE DEMO / SCREENSHOT AREA Switch to your DVWA browser screen now for the demonstration. ⚠️ Safety Note: Use authorized local lab environments only Speaker Notes & Interactivity • Live Demo Steps: Submit comment, refresh page, show how popup triggers automatically every time. • Concept Check: Reiterate alert box is purely a harmless proof-of-concept. Practical Demo Let's See XSS — Safely Practical Demo Let's See XSS — Safely 1. Open DVWA Lab Set Security Level = LOW 2. Open XSS (Stored) Locate guestbook comment form 3. Inject Payload 4. Observe Behavior Popup executes on page load LIVE DEMO / SCREENSHOT AREA Switch to your DVWA browser screen now for the demonstration. ⚠️ Safety Note: Use authorized local lab environments only 1. Open DVWA Lab Set Security Level = LOW 2. Open XSS (Stored) Locate guestbook comment form 3. Inject Payload 4. Observe Behavior Popup executes on page load LIVE DEMO / SCREENSHOT AREA Switch to your DVWA browser screen now for the demonstration. ⚠️ Safety Note: Use authorized local lab environments only 1. Open DVWA Lab Set Security Level = LOW 2. Open XSS (Stored) Locate guestbook comment form 3. Inject Payload 4. Observe Behavior Popup executes on page load 1. Open DVWA Lab Set Security Level = LOW 2. Open XSS (Stored) Locate guestbook comment form 3. Inject Payload 4. Observe Behavior Popup executes on page load LIVE DEMO / SCREENSHOT AREA Switch to your DVWA browser screen now for the demonstration. ⚠️ Safety Note: Use authorized local lab environments only ⚠️ Safety Note: Use authorized local lab environments only Speaker Notes & Interactivity • Live Demo Steps: Submit comment, refresh page, show how popup triggers automatically every time. • Concept Check: Reiterate alert box is purely a harmless proof-of-concept. Speaker Notes & Interactivity Interactive Quiz Audience Challenge SCENARIO: STORED COMMENT User: Alice | Comment: The website stored this comment. Alice opens the page tomorrow. What happens? (Think about what the browser does when it reads The website stored this comment. Alice opens the page tomorrow. What happens? (Think about what the browser does when it reads The website stored this comment. Alice opens the page tomorrow. What happens? (Think about what the browser does when it reads